```
import requests
import streamlit as st
import sqlite3
import pandas as pd
from datetime import datetime
from textwrap import dedent
# ============================================================
# PAGE CONFIGURATION
# ============================================================

st.set_page_config(
    page_title="DripGuard",
    page_icon="💧",
    layout="wide",
    initial_sidebar_state="expanded"
)

DB_NAME = "dripguard.db"


# ============================================================
# CUSTOM CSS
# ============================================================

st.markdown("""
<style>

.stApp {
    background:
        radial-gradient(circle at top right,
        #092c55 0%,
```

```
        #03152d 35%,
        #020b18 100%);
    color: white;
}

header {
    background: transparent !important;
}

section[data-testid="stSidebar"] {
    background:
        linear-gradient(
            180deg,
            #061c39 0%,
            #031329 100%
        );
    border-right: 1px solid #12365c;
}

section[data-testid="stSidebar"] * {
    color: #dcecff;
}

.main-title {
    font-size: 42px;
    font-weight: 800;
    color: white;
    margin-bottom: 0;
}

.subtitle {
```

```
    font-size: 18px;
    color: #9bb9df;
    margin-top: 0;
}

.top-status {
    background: #062b2b;
    border: 1px solid #0bd99a;
    color: #19e6a4;
    padding: 8px 18px;
    border-radius: 20px;
    text-align: center;
    font-weight: 600;
}

.metric-card {
    background:
        linear-gradient(
            145deg,
            rgba(9, 47, 88, 0.95),
            rgba(4, 25, 51, 0.95)
        );
    border: 1px solid #174a7c;
    border-radius: 14px;
    padding: 20px;
    min-height: 135px;
    box-shadow: 0 8px 25px rgba(0,0,0,0.25);
}

.metric-title {
    color: #9dbbe0;
```

```
    font-size: 16px;
}

.metric-number {
    color: white;
    font-size: 35px;
    font-weight: 800;
    margin-top: 5px;
}

.metric-sub {
    color: #8aa8cc;
    font-size: 14px;
}

.normal-card {
    border: 1px solid #00e09a;
}

.attention-card {
    border: 1px solid #f5b82e;
}

.critical-card {
    border: 1px solid #ff334c;
}

.bed-card {
    background:
        linear-gradient(
            145deg,
```

```
            #082b50,
            #04182f
        );
    border-radius: 14px;
    padding: 18px;
    min-height: 280px;
}

.bed-normal {
    border: 2px solid #00e09a;
}

.bed-attention {
    border: 2px solid #f5b82e;
}

.bed-critical {
    border: 2px solid #ff334c;
}

.bed-title {
    font-size: 22px;
    font-weight: 800;
    color: white;
}

.status-normal {
    background: #08a968;
    color: white;
    padding: 5px 14px;
    border-radius: 20px;
```

```
    font-weight: 700;
    font-size: 13px;
}

.status-attention {
    background: #e9aa1b;
    color: #101010;
    padding: 5px 14px;
    border-radius: 20px;
    font-weight: 700;
    font-size: 13px;
}

.status-critical {
    background: #e82e45;
    color: white;
    padding: 5px 14px;
    border-radius: 20px;
    font-weight: 700;
    font-size: 13px;
}

.info-row {
    border-bottom: 1px solid #12385e;
    padding: 10px 0;
    color: #b9d0ec;
}

.info-value {
    float: right;
    color: white;
```

```
    font-weight: 700;
}

.section-box {
    background:
        linear-gradient(
            145deg,
            rgba(7, 37, 72, .95),
            rgba(3, 20, 40, .95)
        );
    border: 1px solid #164878;
    border-radius: 14px;
    padding: 20px;
}

.alert-critical {
    background: #351421;
    border-left: 5px solid #ff334c;
    padding: 12px;
    border-radius: 8px;
    margin-bottom: 10px;
}

.alert-attention {
    background: #342b13;
    border-left: 5px solid #f5b82e;
    padding: 12px;
    border-radius: 8px;
    margin-bottom: 10px;
}
```

```
.alert-normal {
    background: #0b3028;
    border-left: 5px solid #00e09a;
    padding: 12px;
    border-radius: 8px;
    margin-bottom: 10px;
}

.footer {
    text-align: center;
    color: #6f8eaf;
    margin-top: 30px;
}

</style>
""", unsafe_allow_html=True)


# ============================================================
# DATABASE
# ============================================================

def get_readings():

    conn = sqlite3.connect(DB_NAME)

    query = """
        SELECT
            id,
            bed_id,
            drip_rate AS drops_per_minute,
```

```
            flow_rate,
            bag_volume,
            remaining_time,
            status,
            timestamp
        FROM readings
        ORDER BY id ASC
    """

    df = pd.read_sql_query(query, conn)

    conn.close()

    return df


def get_iv_settings():
    try:
        response = requests.get(
            "http://127.0.0.1:5000/api/settings/B01",
            timeout=5
        )

        if response.status_code == 200:
            result = response.json()

            if result.get("success"):
                return pd.DataFrame([{
                    "bed_id": "B01",
                    "bag_volume": result["bag_volume"],
                    "drip_factor": result["drip_factor"],
```

```
                "target_drip_rate": result["target_drip_rate"],
                "updated_at": result.get("updated_at", "")
            }])

    return pd.DataFrame(columns=[
        "bed_id",
        "bag_volume",
        "drip_factor",
        "target_drip_rate",
        "updated_at"
    ])

except requests.RequestException as e:
    st.error(f"Could not load settings from Flask: {e}")
    return pd.DataFrame(columns=[
        "bed_id",
        "bag_volume",
        "drip_factor",
        "target_drip_rate",
        "updated_at"
    ])

conn = sqlite3.connect(DB_NAME)

query = """
    SELECT
        bed_id,
        bag_volume,
        drip_factor,
        target_drip_rate,
        updated_at
```

```
        FROM iv_settings
        ORDER BY bed_id
    """

    try:
        df = pd.read_sql_query(query, conn)
    except Exception:
        df = pd.DataFrame(
            columns=[
                "bed_id",
                "bag_volume",
                "drip_factor",
                "target_drip_rate",
                "updated_at"
            ]
        )

    conn.close()
    return df


def save_iv_settings(bed_id, bag_volume, drip_factor, target_drip_rate):
    response = requests.post(
        "http://127.0.0.1:5000/api/settings",
        json={
            "bed_id": bed_id,
            "bag_volume": float(bag_volume),
            "drip_factor": float(drip_factor),
            "target_drip_rate": float(target_drip_rate)
        },
        timeout=5
```

```
    )

    response.raise_for_status()
    result = response.json()

    if not result.get("success"):
        raise Exception(result.get("error", "Settings were not saved"))

    return result

# ============================================================
# STATUS
# ============================================================

def get_status(drops):

    drops = float(drops)

    if drops == 0:
        return "CRITICAL"

    elif drops < 10:
        return "ATTENTION"

    else:
        return "NORMAL"


# ============================================================
# SIDEBAR
# ============================================================
```

```
with st.sidebar:

    st.markdown("""
    <div style="
        text-align:center;
        padding:15px;
        font-size:25px;
        font-weight:800;
        color:white;">
        💧 Drip<span style="color:#159cff;">Guard</span>
    </div>
    """, unsafe_allow_html=True)

    st.markdown("---")

    page = st.radio(
        "",
        [
            "🏠 Dashboard",
            "📈 Live Monitoring",
            "🕘 History",
            "🔔 Alerts",
            "🗄️ Database",
            "⚙️ Settings"
        ]
    )

    st.markdown("---")

    st.markdown("""
```

```
    <div style="
        text-align:center;
        color:#7899bd;
        margin-top:180px;">
        ❤️<br>
        Better Monitoring.<br>
        Safer Care.
    </div>
    """, unsafe_allow_html=True)


# ============================================================
# LOAD DATA
# ============================================================

try:

    df = get_readings()

except Exception as e:

    st.error(f"Database error: {e}")
    st.stop()


# ============================================================
# HEADER
# ============================================================

col1, col2 = st.columns([4, 1])
```

```
with col1:

    st.markdown(
        '<div class="main-title">💧 DripGuard</div>',
        unsafe_allow_html=True
    )

    st.markdown(
        '<div class="subtitle">'
        'Smart IV Infusion Monitoring System'
        '</div>',
        unsafe_allow_html=True
    )

with col2:

    st.markdown(
        '<div class="top-status">🟢 Connected</div>',
        unsafe_allow_html=True
    )


st.divider()


# ============================================================
# DASHBOARD PAGE
# ============================================================

if page == "🏠 Dashboard":
```

```
st.markdown(
    '<h1 style="font-size:32px;">IV Infusion Dashboard</h1>',
    unsafe_allow_html=True
)

st.markdown(
    '<p style="color:#9bb9df;font-size:17px;">'
    'Real-time monitoring of multiple IV units'
    '</p>',
    unsafe_allow_html=True
)


# ----------------------------------------------------
# NO DATA
# ----------------------------------------------------

if df.empty:

    st.warning(
        "No ESP8266 readings have been received yet."
    )

    st.info(
        "Connect your ESP8266 and send readings "
        "to the Flask server."
    )

    st.stop()
```

```
# ------------------------------------------------
# LATEST BED DATA
# ------------------------------------------------

latest = (
    df.sort_values("timestamp")
      .groupby("bed_id")
      .tail(1)
      .copy()
)

latest["status"] = latest[
    "drops_per_minute"
].apply(get_status)


total = len(latest)

normal = len(
    latest[latest["status"] == "NORMAL"]
)

attention = len(
    latest[latest["status"] == "ATTENTION"]
)

critical = len(
    latest[latest["status"] == "CRITICAL"]
)
```

```
# ------------------------------------------------
# TOP METRICS
# ------------------------------------------------

c1, c2, c3, c4 = st.columns(4)

with c1:

    st.markdown(f"""
    <div class="metric-card">

    <div class="metric-title">
    💧 Total IVs
    </div>

    <div class="metric-number">
    {total}
    </div>

    <div class="metric-sub">
    Beds Monitored
    </div>

    </div>
    """, unsafe_allow_html=True)


with c2:

    st.markdown(f"""
    <div class="metric-card normal-card">
```

```
    <div class="metric-title"
    style="color:#00e09a;">
    ✔ Normal
    </div>

    <div class="metric-number">
    {normal}
    </div>

    <div class="metric-sub">
    IVs
    </div>

    </div>
    """, unsafe_allow_html=True)


with c3:

    st.markdown(f"""
    <div class="metric-card attention-card">

    <div class="metric-title"
    style="color:#f5b82e;">
    ⚠ Attention
    </div>

    <div class="metric-number">
    {attention}
    </div>
```

```
    <div class="metric-sub">
    IVs
    </div>

    </div>
    """, unsafe_allow_html=True)


with c4:

    st.markdown(f"""
    <div class="metric-card critical-card">

    <div class="metric-title"
    style="color:#ff334c;">
    🚨 Critical
    </div>

    <div class="metric-number">
    {critical}
    </div>

    <div class="metric-sub">
    IVs
    </div>

    </div>
    """, unsafe_allow_html=True)
```

```
st.markdown("### 🛏️ IV Bed Status")


# ----------------------------------------------------
# BED CARDS
# ----------------------------------------------------

bed_columns = st.columns(4)

for index, (_, row) in enumerate(
    latest.iterrows()
):

    bed = row["bed_id"]

    drops = float(
        row["drops_per_minute"]
    )

    status = row["status"]


    if status == "NORMAL":

        card_class = "bed-normal"
        status_class = "status-normal"
        status_text = "NORMAL"
        message = "🟢 Flowing Normally"

    elif status == "ATTENTION":
```

```
    card_class = "bed-attention"
    status_class = "status-attention"
    status_text = "ATTENTION"
    message = "🟡 Low Drip Rate"

else:

    card_class = "bed-critical"
    status_class = "status-critical"
    status_text = "CRITICAL"
    message = "🔴 No Drops Detected"


# Approximate flow rate
flow_rate = drops * 0.05
bag_volume = float(row["bag_volume"])

# Use the saved configuration for B01 when available.
if bed == "B01":
    current_settings = get_iv_settings()
    if not current_settings.empty:
        bag_volume = float(current_settings.iloc[0]["bag_volume"])

bag_volume_display = str(bag_volume)
if bag_volume_display.endswith(".0"):
    bag_volume_display = bag_volume_display[:-2]

# Calculate approximate remaining time.
if flow_rate > 0:
    remaining = bag_volume / flow_rate
    remaining_text = f"{remaining:.0f} min"
```

```
else:
    remaining_text = "STOPPED"

with bed_columns[index]:

    st.html(dedent(f"""

    <div class="bed-card {card_class}">

        <div style="display:flex; justify-content:space-between; align-items:center;">

            <div class="bed-title">
            🛏️ {bed}
            </div>

            <span class="{status_class}">
            {status_text}
            </span>

        </div>

        <div class="info-row">
            💧 Drip Rate
            <span class="info-value">
            {drops:.0f} drops/min
            </span>
        </div>

        <div class="info-row">
            ≈ Flow Rate
            <span class="info-value">
```

```
            {flow_rate:.2f} mL/min
            </span>
        </div>

        <div class="info-row">
            ⏱ Remaining Time
            <span class="info-value">
            {remaining_text}
            </span>
        </div>

        <div class="info-row">
            💉 Bag Volume
            <span class="info-value">
                        {bag_volume_display} mL
            </span>
        </div>

        <br>

        <b>{message}</b>

    </div>

    """))


st.markdown("###")


# ----------------------------------------------------
```

```
# BOTTOM SECTION
# ----------------------------------------------------

chart_col, table_col, alert_col = st.columns(
    [1.4, 1.1, 1]
)


# ----------------------------------------------------
# DRIP RATE GRAPH
# ----------------------------------------------------

with chart_col:

    st.markdown(
        '<div class="section-box">',
        unsafe_allow_html=True
    )

    st.markdown(
        "### 📈 Drip Rate Trend"
    )

    st.caption(
        "Live readings from ESP8266"
    )

    chart_df = df.copy()

    chart_df["timestamp"] = pd.to_datetime(
        chart_df["timestamp"]
```

```
    )

    chart_df = chart_df.pivot_table(
        index="timestamp",
        columns="bed_id",
        values="drops_per_minute",
        aggfunc="last"
    )

    st.line_chart(
        chart_df,
        height=300
    )

    st.markdown(
        "</div>",
        unsafe_allow_html=True
    )


# ----------------------------------------------------
# RECENT READINGS
# ----------------------------------------------------

with table_col:

    st.markdown(
        '<div class="section-box">',
        unsafe_allow_html=True
    )
```

```
st.markdown(
    "### 📋 Recent Readings"
)

recent = df.tail(8).copy()

recent["Status"] = recent[
    "drops_per_minute"
].apply(get_status)

recent = recent.rename(
    columns={
        "bed_id": "Bed",
        "drops_per_minute": "Drops/min",
        "timestamp": "Time"
    }
)

st.dataframe(
    recent[
        [
            "Time",
            "Bed",
            "Drops/min",
            "Status"
        ]
    ].iloc[::-1],
    use_container_width=True,
    hide_index=True
)
```

```
    st.markdown(
        "</div>",
        unsafe_allow_html=True
    )


# ----------------------------------------------------
# ALERTS
# ----------------------------------------------------

with alert_col:

    st.markdown(
        '<div class="section-box">',
        unsafe_allow_html=True
    )

    st.markdown(
        "### 🔔 Latest Alerts"
    )

    alert_data = latest[
        latest["status"] != "NORMAL"
    ].sort_values(
        "timestamp",
        ascending=False
    )

    if alert_data.empty:

        st.success(
```

```
        "No active alerts."
    )

else:

    for _, alert in alert_data.iterrows():

        bed = alert["bed_id"]

        drops = float(
            alert["drops_per_minute"]
        )

        status = alert["status"]

        time = alert["timestamp"]

        if status == "CRITICAL":

            st.markdown(f"""
            <div class="alert-critical">
            <b>🚨 {bed} - Critical</b><br>
            No drops detected.<br>
            <small>{time}</small>
            </div>
            """,
            unsafe_allow_html=True)

        else:

            st.markdown(f"""
```

```
                <div class="alert-attention">
                <b>⚠️ {bed} - Attention</b><br>
                Low drip rate
                ({drops:.0f} drops/min).<br>
                <small>{time}</small>
                </div>
                """,
                unsafe_allow_html=True)


        st.markdown(
            "</div>",
            unsafe_allow_html=True
        )


# ============================================================
# LIVE MONITORING
# ============================================================

elif page == "📈 Live Monitoring":

    st.title("📈 Live Monitoring")

    if df.empty:

        st.warning("No ESP8266 data available.")

    else:

        latest = (
```

```
            df.sort_values("timestamp")
              .groupby("bed_id")
              .tail(1)
        )

        st.dataframe(
            latest[
                [
                    "bed_id",
                    "drops_per_minute",
                    "timestamp"
                ]
            ],
            use_container_width=True,
            hide_index=True
        )


# ============================================================
# HISTORY
# ============================================================

elif page == "🕘 History":

    st.title("🕘 Reading History")

    if df.empty:

        st.info("No historical readings yet.")

    else:
```

```
        st.dataframe(
            df.sort_values(
                "timestamp",
                ascending=False
            ),
            use_container_width=True,
            hide_index=True
        )


# ========================================================
# ALERTS
# ========================================================

elif page == "🔔 Alerts":

    st.title("🔔 Alerts")

    if df.empty:

        st.info("No alerts.")

    else:

        alerts = df[
            df["drops_per_minute"] < 10
        ].copy()

        if alerts.empty:
```

```
            st.success(
                "No critical or attention readings."
            )

        else:

            alerts["Status"] = alerts[
                "drops_per_minute"
            ].apply(get_status)

            st.dataframe(
                alerts.sort_values(
                    "timestamp",
                    ascending=False
                ),
                use_container_width=True,
                hide_index=True
            )


# ========================================================
# DATABASE
# ========================================================

elif page == "🗄 Database":

    st.title("🗄 Database")

    st.write(
        f"Database: `{DB_NAME}`"
    )
```

```
    st.metric(
        "Total Stored Readings",
        len(df)
    )

    if not df.empty:

        st.dataframe(
            df,
            use_container_width=True,
            hide_index=True
        )


# ============================================================
# SETTINGS
# ============================================================

elif page == "⚙️ Settings":

    st.title("⚙️ Settings")

    st.write(
        "DripGuard System Configuration"
    )

    st.text_input(
        "Hospital / Project Name",
        "DripGuard"
    )
```

```
    st.number_input(
        "Warning threshold (drops/min)",
        min_value=1,
        max_value=50,
        value=10
    )

    st.number_input(
        "Critical threshold (drops/min)",
        min_value=0,
        max_value=10,
        value=0
    )

    st.success(
        "Settings page ready."
    )


# ============================================================
# IV CONFIGURATION
# ============================================================

st.markdown("---")
st.markdown("## ⚙️ IV Configuration")
st.caption("Save infusion settings for each Bed ID in the SQLite database.")

settings_df = get_iv_settings()

config_beds = sorted(set(settings_df["bed_id"].tolist())) if not settings_df.empty else []
```

```
default_bed = config_beds[0] if config_beds else "B01"

cfg1, cfg2, cfg3, cfg4 = st.columns(4)

with cfg1:
    config_bed_id = st.text_input("Bed ID", value=default_bed, key="config_bed_id")

existing = settings_df[settings_df["bed_id"] == config_bed_id.strip()] if not settings_df.empty else pd.DataFrame()

if not existing.empty:
    existing_row = existing.iloc[0]
    default_volume = float(existing_row["bag_volume"])
    default_factor = float(existing_row["drip_factor"])
    default_target = float(existing_row["target_drip_rate"])
else:
    default_volume = 300.0
    default_factor = 20.0
    default_target = 30.0

with cfg2:
    config_bag_volume = st.number_input(
        "Bag Volume (mL)",
        min_value=1.0,
        value=default_volume,
        step=10.0,
        key="config_bag_volume"
    )

with cfg3:
    config_drip_factor = st.number_input(
        "Drip Factor (drops/mL)",
```

```
        min_value=0.1,
        value=default_factor,
        step=1.0,
        key="config_drip_factor"
    )

with cfg4:
    config_target_rate = st.number_input(
        "Target Drip Rate (drops/min)",
        min_value=0.1,
        value=default_target,
        step=1.0,
        key="config_target_rate"
    )

if st.button("Apply Settings", type="primary", key="apply_iv_settings"):
    if not config_bed_id.strip():
        st.error("Please enter a Bed ID.")
    else:
        try:
            save_iv_settings(
                config_bed_id.strip(),
                float(config_bag_volume),
                float(config_drip_factor),
                float(config_target_rate)
            )
            st.success(
                f"Settings saved for {config_bed_id.strip()} in SQLite."
            )
            st.rerun()
        except Exception as e:
```

```
            st.error(f"Could not save settings: {e}")

st.markdown("### Saved IV Configurations")

saved_settings = get_iv_settings()

if saved_settings.empty:
    st.info("No IV configurations saved yet.")
else:
    st.dataframe(
        saved_settings,
        use_container_width=True,
        hide_index=True
    )


# ============================================================
# FOOTER
# ============================================================

st.markdown(
    """
    <div class="footer">
    ❤️ Better Monitoring. Safer Care. |
    DripGuard Smart IV Infusion Monitoring System
    </div>
    """,
    unsafe_allow_html=True
)
```